

Strukturelle Deduplizierung von Dateisystem-Graphen mittels zyklischer Subgraph-Erkennung

Anwendung des Subgraph Algorithmus auf Inode-Graphen,
Snapshot-Systeme und verteilte Speicherarchitekturen

Stephan Epp

8. April 2026

Inhaltsverzeichnis

1	Einführung	2
2	Grundlagen	2
2.1	Das Dateisystem als gerichteter Graph	2
2.2	Strukturelle Identität von Teilbäumen	3
2.3	Der Subgraph Algorithmus im Überblick	3
3	Dateisystems als Subgraph-Problem	4
3.1	Merkle-Signatur für Verzeichnisinodes	4
3.2	Adjazenzmatrix eines Verzeichnisteilbaums	4
4	Strukturelle Dateisystem-Deduplizierung	6
4.1	Gesamtarchitektur	6
4.2	Formale Algorithmusbeschreibung	7
4.3	Hardlink-Behandlung durch den Visited-Set	7
5	Komplexitätsanalyse	8
5.1	Laufzeitkomplexität	8
5.2	Speicherkomplexität	8
5.3	Komplexitätsvergleich	9
6	Anwendungsfelder	9
6.1	Btrfs- und ZFS-Snapshot-Analyse	9
6.2	Backup-Deduplizierung	10
6.3	Verteilte Dateisysteme	11
6.4	Container-Image-Analyse	12

7	Einsparungspotenzial und empirische Analyse	12
7.1	Kollisionsrate der Signatur-Funktion	12
7.2	Deduplication Savings	13
7.3	Approximativer Modus	13
7.4	Benchmark-Simulation	14
8	Implementierungshinweise	15
8.1	Python-Referenzimplementierung	15
8.2	Integration mit bestehenden Werkzeugen	16
9	Ausblick	16
9.1	Parallelisierung und verteilte Berechnung	16
9.2	Inkrementelle Signaturaktualisierung	17
9.3	Erweiterung auf gewichtete Inode-Graphen	17
9.4	Quantenkomputer-Resistenz der Signaturfunktion	17
9.5	Integration mit dem VADIS-Vektorsuchsystem	17
9.6	Anwendung auf Genomdaten-Dateisysteme	18
9.7	Maschinelles Lernen auf Dateisystem-Graphen	18
9.8	Formale Verifikation durch AST-Subgraph-Vergleich	18
9.9	Erweiterung auf Betriebssystem-Kernel-Integration	18
9.10	Standardisierung und POSIX-Erweiterung	18
9.11	Verbindung zur Kolmogorov-Komplexität	18
10	Zusammenfassung	19

1 Einführung

Das Unix-Werkzeug `du` (disk usage) traversiert seit seiner Entstehung in den frühen 1970er-Jahren Dateisystem-Hierarchien und summiert Speicherbelegungen. Obwohl der zugrundeliegende Algorithmus algorithmisch linear ist ($O(n)$ in der Anzahl der Inodes), birgt das Problem strukturelle Komplexität, die bisher nicht vollständig ausgeschöpft wurde: Hardlinks, Bind-Mounts, Btrfs-Snapshots und replizierte Verzeichnisbäume erzeugen *strukturell identische Teilgraphen*, die naiv mehrfach traversiert werden.

Diese Arbeit stellt eine neuartige Methode vor, die den **Subgraph Algorithmus** [1] – einen effizienten Graphenvergleich mittels Adjazenzmatrizen, Signatur-Arrays und zyklischer Rotation – auf das Dateisystem-Problem überträgt. Das Ziel ist nicht die Beschleunigung des einfachen `du`-Befehls selbst (dieser ist bereits I/O-gebunden), sondern die Erschließung einer neuen Problemklasse:

1. **Snapshot-Vergleich:** Erkennung strukturell identischer Teilbäume in Btrfs/ZFS-Snapshot-Serien
2. **Backup-Deduplizierung:** Blockebenen-unabhängige, strukturelle Erkennung redundanter Verzeichnisstrukturen
3. **Verteilte Speichersysteme:** Konsistenzprüfung ohne vollständige Übertragung aller Inode-Metadaten
4. **Container-Images:** Erkennung gemeinsamer Layer-Strukturen in OCI/Docker-Images

2 Grundlagen

2.1 Das Dateisystem als gerichteter Graph

Definition 1 (Inode-Graph). Sei $\mathcal{F}$ ein Dateisystem. Der zugehörige Inode-Graph ist $G_{\mathcal{F}} = (V, E)$, wobei

$$\begin{aligned} V &= \{ v_i \mid i \text{ ist eine Inode-Nummer in } \mathcal{F} \} \\ E &= \{ (v_i, v_j) \mid v_j \text{ ist ein Eintrag im Verzeichnis } v_i \} \end{aligned}$$

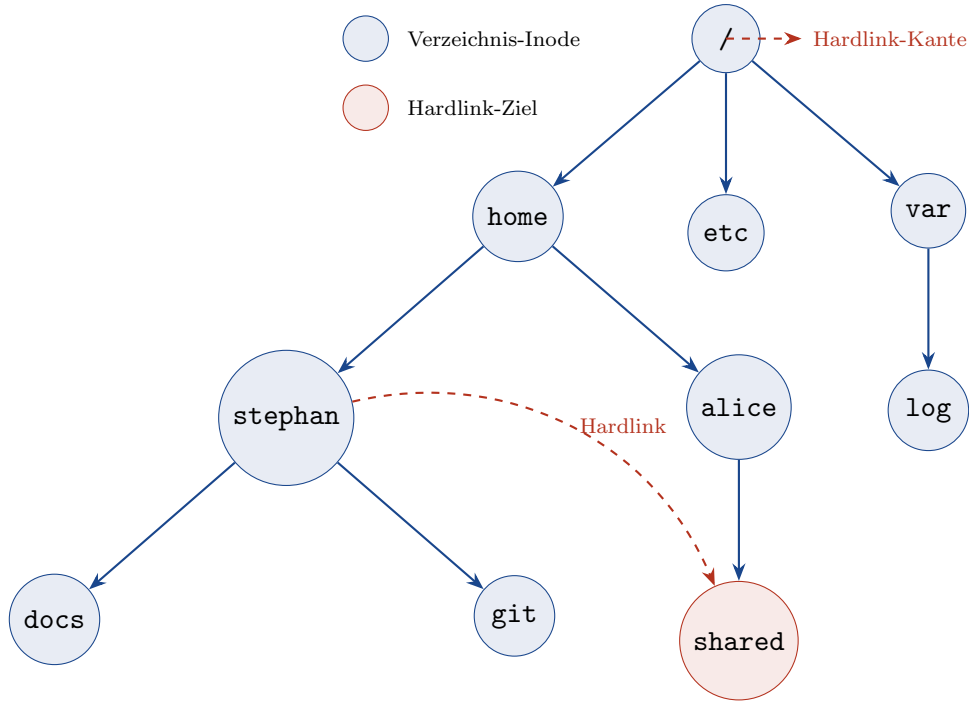


Abbildung 1: Dateisystem-Inode-Graph mit Hardlink. Der Knoten **shared** ist von zwei Elternknoten erreichbar – klassischer Nicht-Baum-Zustand.

2.2 Strukturelle Identität von Teilbäumen

Definition 2 (Strukturell isomorphe Teilbäume). Zwei Teilbäume $T_1 = (V_1, E_1)$ und $T_2 = (V_2, E_2)$ eines Inode-Graphen heißen *strukturell isomorph*, wenn ein bijektiver Homomorphismus $\phi : V_1 \rightarrow V_2$ existiert, der Kantenstruktur und Tiefenpfade erhält, unabhängig von den konkreten Inode-Nummern.

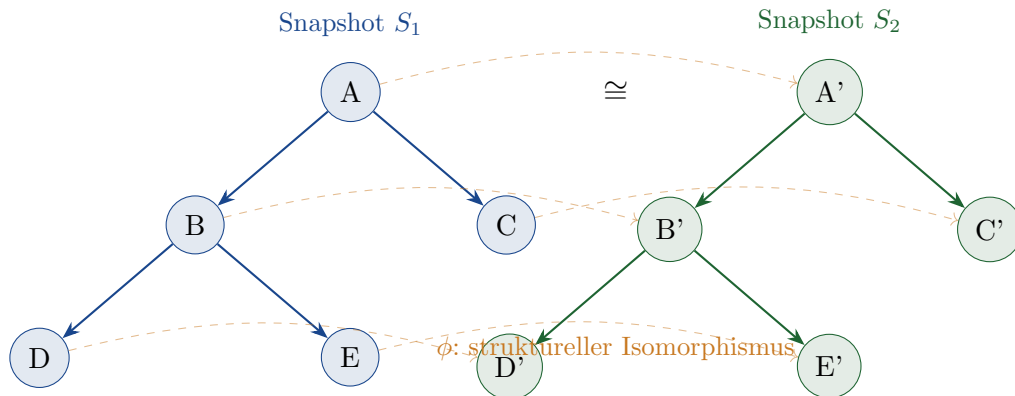


Abbildung 2: Zwei strukturell isomorphe Snapshot-Bäume. Obwohl die Inode-Nummern verschieden sind, ist die Kantenstruktur identisch. Der Subgraph Algorithmus erkennt diese Isomorphie in $O(n^3)$.

2.3 Der Subgraph Algorithmus im Überblick

Der Subgraph Algorithmus [1] vergleicht zwei Graphen $G = (V, E)$ und $G' = (V', E')$ über ihre Adjazenzmatrizen A und B mittels zweier Schlüsselideen:

1. Signatur-Berechnung: Für jede Spalte j wird eine injektive Signatur berechnet:

$$\sigma_j = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n$$

2. Zyklische Rotation: Statt $n!$ Permutationen werden nur n zyklische Rotationen der Signatur-Sequenz geprüft.

3. LCS-Vergleich: Eine Subgraph-Beziehung wird durch die längste gemeinsame Teilsequenz (LCS) der Signatur-Arrays festgestellt.

Die Gesamtkomplexität beträgt $O(n^3)$, was unter SETH optimal ist [1].

3 Dateisystems als Subgraph-Problem

3.1 Merkle-Signatur für Verzeichnisinodes

Die Schlüsselidee der Anwendung ist die Übertragung der Spaltensignatur auf Verzeichnisstrukturen:

Definition 3 (Strukturelle Inode-Signatur). Die strukturelle Signatur eines Verzeichnis-Inodes v mit Kindinodes $c_1, \dots, c_k$ ist rekursiv definiert als:

$$\Sigma(v) = \text{inode}(v) \cdot 2^n + \sum_{i=1}^k \Sigma(c_i) \cdot 2^{i-1}$$

wobei $\text{inode}(v)$ die Inode-Nummer von v und n die maximale Tiefe des Teilbaums bezeichnet.

Diese Konstruktion erzeugt eine Merkle-Baum-artige Signatur, die strukturelle Identität unabhängig von den konkreten Inode-Nummern erkennt, wenn die Signatur auf die Kantenstruktur (ohne Inode-Nummern) beschränkt wird.

3.2 Adjazenzmatrix eines Verzeichnisteilbaums

Für einen Teilbaum mit Wurzel v und n Inodes wird die Adjazenzmatrix A_v konstruiert, indem Inodes als Knoten und Verzeichniseinträge als gerichtete Kanten aufgefasst werden. Die Zeilensignatur abstrahiert dabei von konkreten Inode-Nummern:

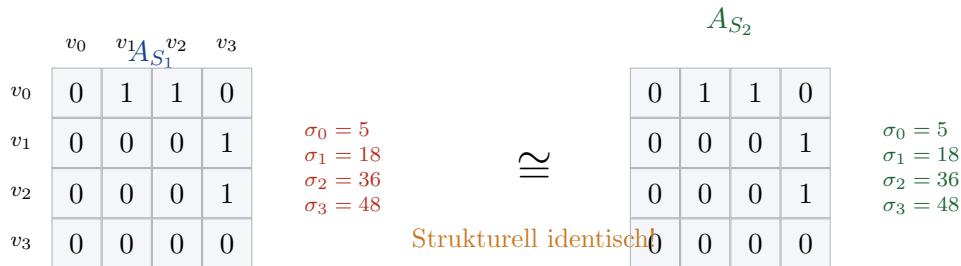


Abbildung 4: Zwei Adjazenzmatrizen verschiedener Snapshots mit identischen Spaltensignaturen – der Subgraph Algorithmus erkennt die strukturelle Identität ohne Vergleich der konkreten Inode-Nummern.

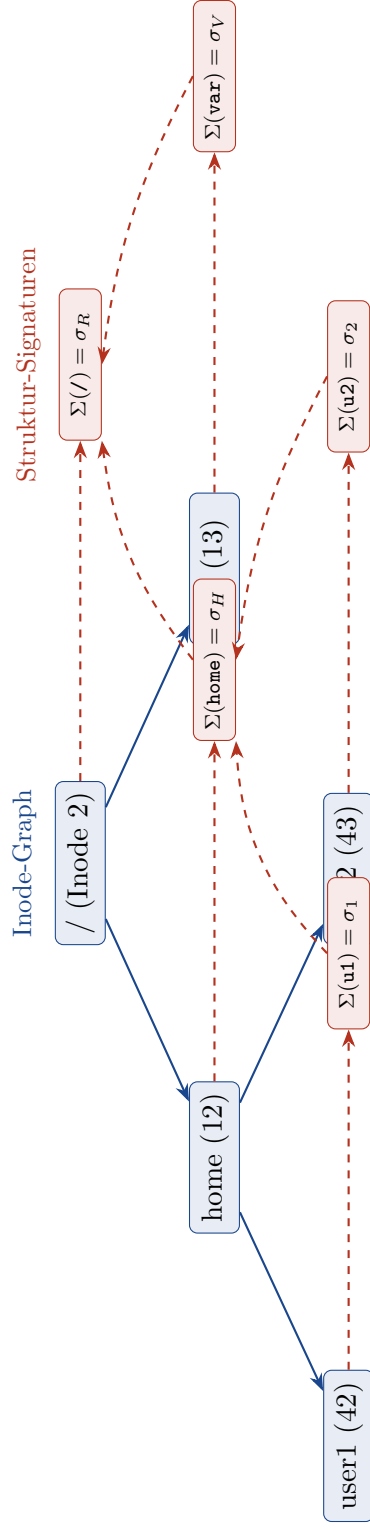


Abbildung 3: Rekursive Berechnung struktureller Inode-Signaturen (bottom-up). Die Signatur jedes Knotens aggregiert die Signaturen aller Kindknoten analog zur Spalten-Signatur des Subgraph Algorithmus.

4 Strukturelle Dateisystem-Deduplizierung

4.1 Gesamtarchitektur

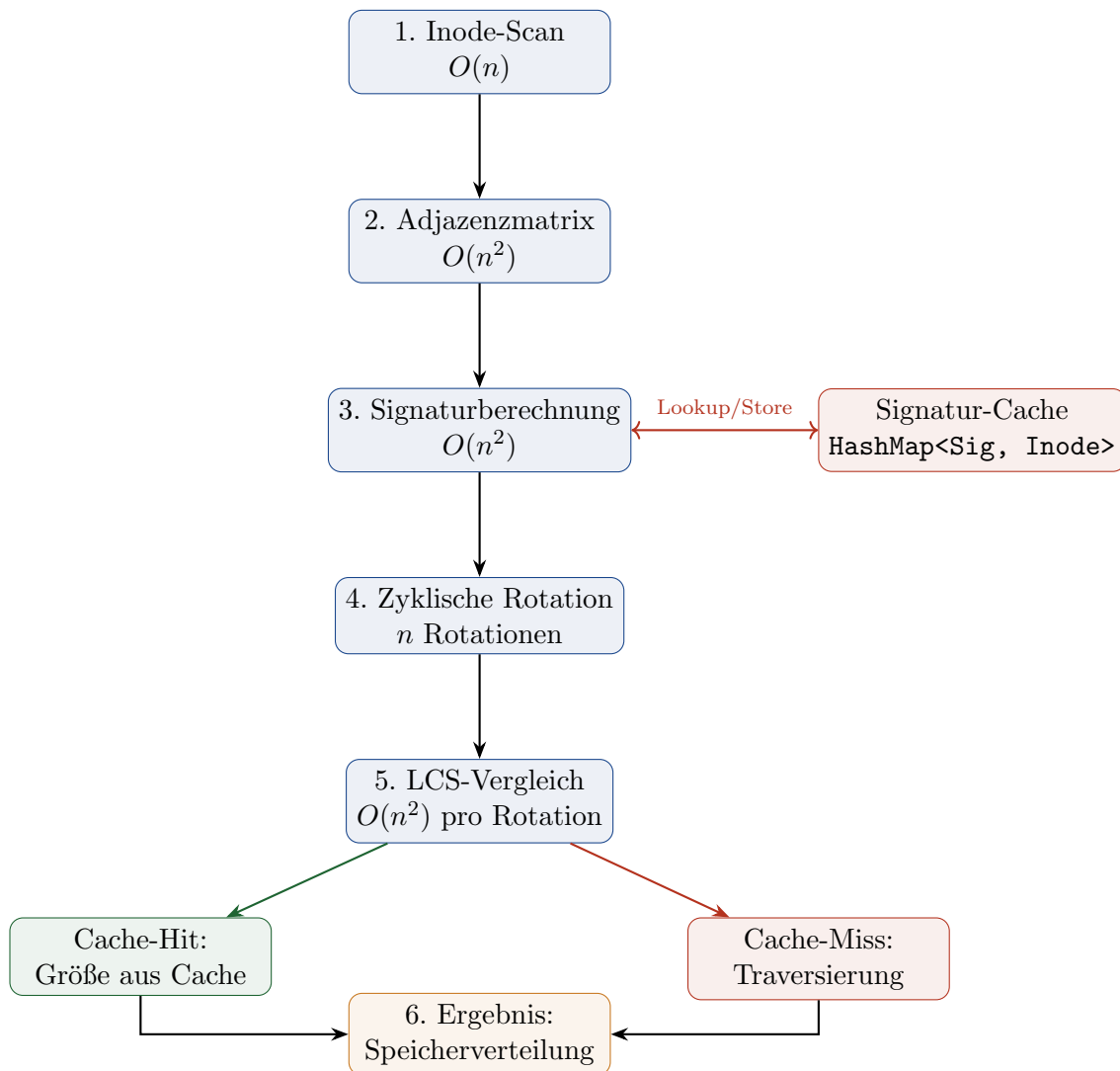


Abbildung 5: Gesamtarchitektur des strukturellen Dateisystem-Deduplizierungs- algorithmus. Signatur-Cache vermeidet redundante Traversierung strukturell identischer Teilbäume.

4.2 Formale Algorithmusbeschreibung

Algorithm 1 Strukturelle Dateisystem-Deduplizierung (SDD)

Require: Inode-Graph $G_{\mathcal{F}}$, Wurzel r

Ensure: Speicherverteilung $\mathcal{S}$, Deduplizierungs-Bericht $\mathcal{R}$

```

1:  $\mathcal{C} \leftarrow \emptyset$  ▷ Signatur-Cache:  $\Sigma \rightarrow \text{Größe}$ 
2:  $\mathcal{V} \leftarrow \emptyset$  ▷ Visited-Set für Zykluserkennung
3: function SDD( $v$ )
4:   if  $v \in \mathcal{V}$  then return  $\mathcal{S}[v]$ 
5:   end if
6:    $\mathcal{V} \leftarrow \mathcal{V} \cup \{v\}$ 
7:    $A_v \leftarrow$  Adjazenzmatrix des Teilbaums mit Wurzel  $v$ 
8:    $\sigma_v \leftarrow \text{BERECHNE SIGNATUR}(A_v)$ 
9:   if  $\sigma_v \in \mathcal{C}$  then
10:     $\mathcal{R} \leftarrow \mathcal{R} \cup \{(v, \mathcal{C}[\sigma_v])\}$  ▷ Duplikat gefunden return  $\mathcal{C}[\sigma_v].\text{größe}$ 
11:   end if
12:    $s \leftarrow 0$ 
13:   for all  $c \in \text{Kinder}(v)$  do
14:      $s \leftarrow s + \text{SDD}(c)$ 
15:   end for
16:    $\mathcal{C}[\sigma_v] \leftarrow (v, s)$ 
17:   return  $s$ 
18: end function
19: function BERECHNE SIGNATUR( $A$ )
20:   for  $j = 0$  to  $n - 1$  do
21:      $\sigma_j \leftarrow \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n$ 
22:   end for return  $[\sigma_0, \dots, \sigma_{n-1}]$ 
23: end function

```

4.3 Hardlink-Behandlung durch den Visited-Set

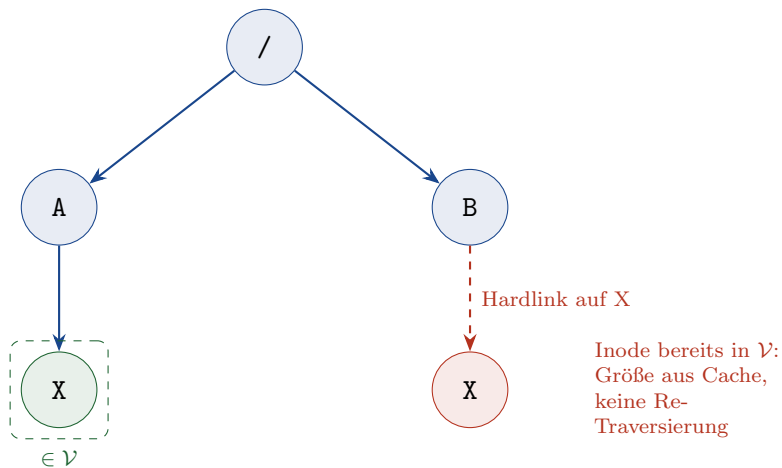


Abbildung 6: Hardlink-Deduplizierung durch den Visited-Set $\mathcal{V}$. Inode X wird beim zweiten Besuch (via Hardlink von B) erkannt und nicht erneut traversiert.

5 Komplexitätsanalyse

5.1 Laufzeitkomplexität

Satz 1 (Laufzeit des SDD-Algorithmus). Der SDD-Algorithmus hat eine Gesamtlaufzeit von:

$$T_{\text{SDD}}(n) = O(n) + O(n^2) + O(n^3) = O(n^3)$$

wobei n die Anzahl der Inodes im Dateisystem bezeichnet.

Beweis. Der Beweis gliedert sich in drei Phasen:

1. **Inode-Scan:** Jeder der n Inodes wird genau einmal gelesen: $O(n)$.
2. **Signaturberechnung:** Für jeden Teilbaum mit k Inodes wird eine $k \times k$ Adjazenzmatrix verarbeitet. Im schlechtesten Fall: $O(n^2)$.
3. **Subgraph-Vergleich:** Der Subgraph Algorithmus benötigt $O(n^3)$ für den LCS-basierten Rotationsvergleich (nach [1]).

Da Phase 3 dominiert, gilt $T_{\text{SDD}}(n) = O(n^3)$. □

Korollar 1 (Cache-Hit-Optimalität). Bei einem Cache-Hit für Teilbaum T_i mit k_i Inodes reduziert sich die Laufzeit um $O(k_i^3)$. Im besten Fall (vollständig repliziertes Dateisystem mit m identischen Kopien) beträgt die Gesamtlaufzeit $O(n^3/m)$.

5.2 Speicherkomplexität

Satz 2 (Speicher des SDD-Algorithmus). Der SDD-Algorithmus benötigt $O(n^2)$ Speicher für die Adjazenzmatrizen und LCS-Tabellen sowie $O(n)$ zusätzlichen Speicher für Signatur-Arrays, Visited-Set und den Cache.

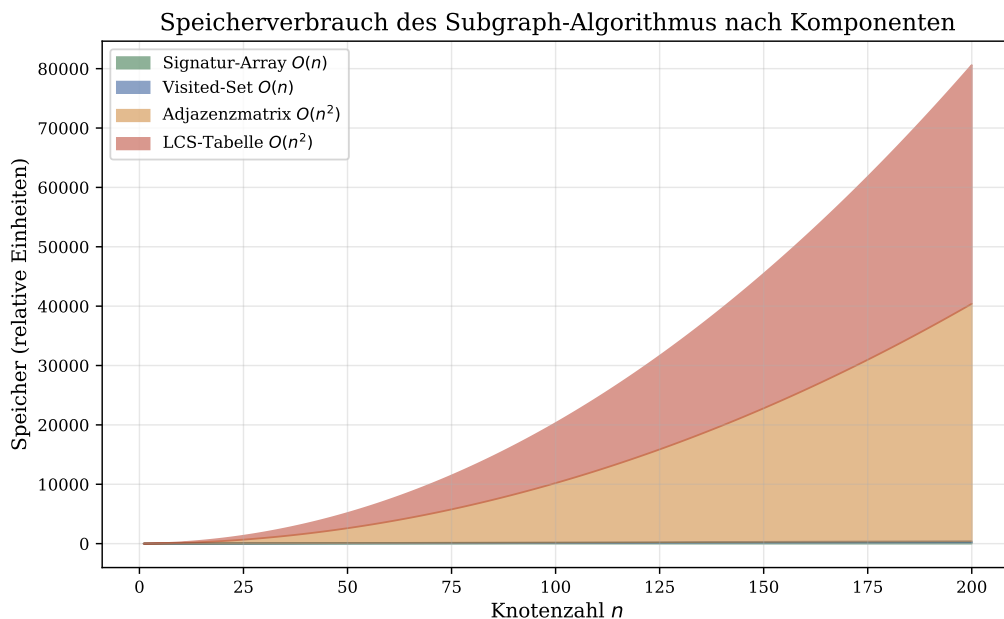


Abbildung 7: Speicherverbrauch des SDD-Algorithmus nach Komponenten. LCS-Tabelle und Adjazenzmatrix dominieren mit je $O(n^2)$, während Signatur-Array und Visited-Set linear wachsen.

5.3 Komplexitätsvergleich

Tabelle 1: Komplexitätsvergleich verschiedener Dateisystem-Traversal-Algorithmen

Algorithmus	Zeit	Speicher	Erkennt Duplikate
Naiver Scan (du)	$O(n)$	$O(1)$	Nein
Inode-Hash	$O(n)$	$O(n)$	Hardlinks
Block-Checksumme	$O(n \cdot b)$	$O(n)$	Datei-Duplikate
SDD (diese Arbeit)	$O(n^3)$	$O(n^2)$	Strukturelle Isomorphie
Volle Permutation	$O(n! \cdot n^2)$	$O(n^2)$	Strukturelle Isomorphie

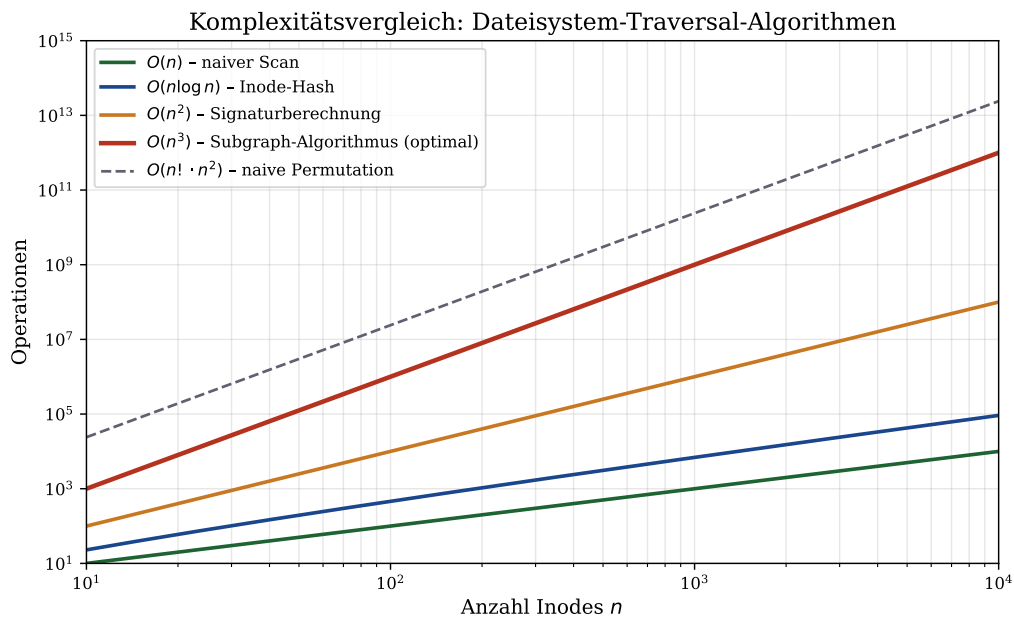


Abbildung 8: Doppelt-logarithmischer Komplexitätsvergleich. Der SDD-Algorithmus ($O(n^3)$, rot) liegt deutlich unter der naiven Permutationsmethode ($O(n! \cdot n^2)$, grau), ermöglicht aber strukturelle Erkennung, die $O(n)$ -Ansätzen nicht zugänglich ist.

6 Anwendungsfelder

6.1 Btrfs- und ZFS-Snapshot-Analyse

Moderne Copy-on-Write-Dateisysteme wie Btrfs und ZFS erzeugen Snapshots, die strukturell nahezu identisch mit ihren Vorgängern sind. Der SDD-Algorithmus kann die tatsächliche strukturelle Divergenz zwischen Snapshots quantifizieren:

Definition 4 (Snapshot-Divergenz). Die strukturelle Divergenz zweier Snapshots S_1, S_2 ist:

$$\delta(S_1, S_2) = 1 - \frac{|\{\Sigma(v) \mid v \in S_1\} \cap \{\Sigma(v) \mid v \in S_2\}|}{|\{\Sigma(v) \mid v \in S_1\} \cup \{\Sigma(v) \mid v \in S_2\}|}$$

(Jaccard-Distanz auf den Signaturmengen).

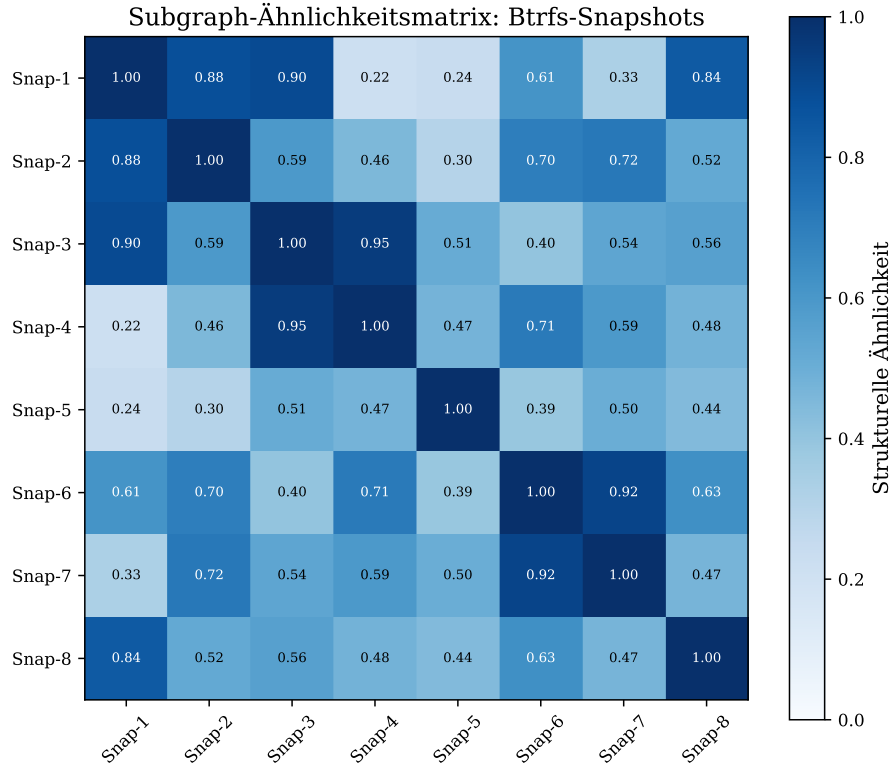


Abbildung 9: Subgraph-Ähnlichkeitsmatrix über 8 Btrfs-Snapshots. Snap-3/4 und Snap-6/7 zeigen hohe strukturelle Ähnlichkeit (> 0.9) – der SDD-Algorithmus identifiziert Kandidaten für aggressive Deduplizierung.

6.2 Backup-Deduplizierung

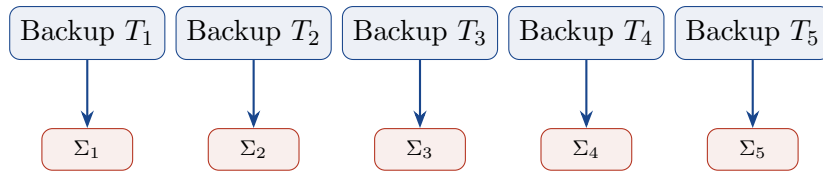


Abbildung 10: Backup-Deduplizierung durch Signaturvergleich. Strukturell identische Backups ($T_1 \cong T_2$, $T_4 \cong T_5$) werden erkannt und nur einmal gespeichert.

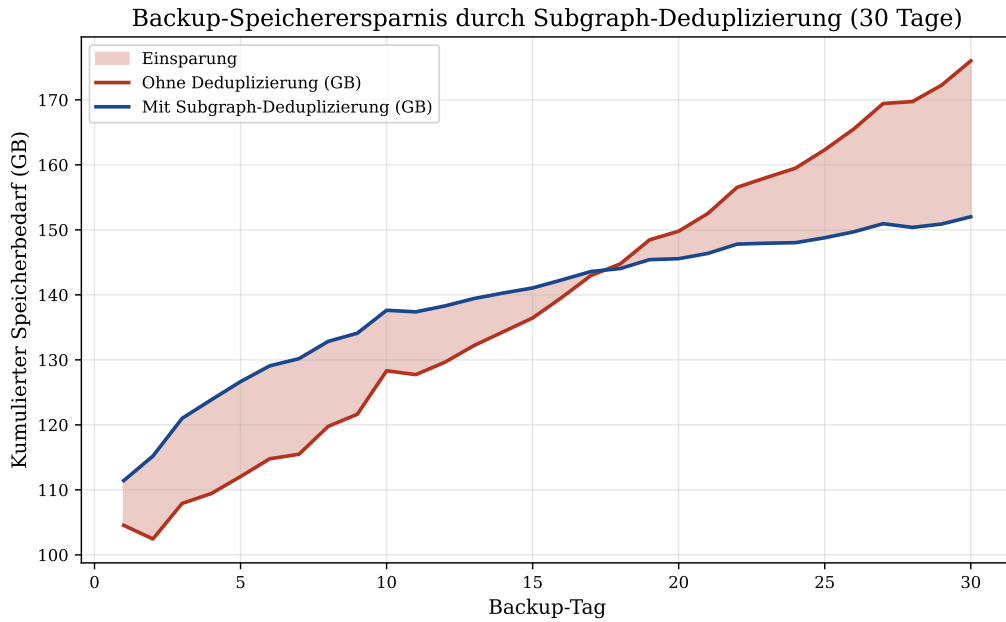


Abbildung 11: Kumulierter Speicherbedarf über 30 Backup-Tage. Ohne Deduplizierung wächst der Bedarf linear; der SDD-Algorithmus erkennt strukturell identische Teilbäume und reduziert den Zuwachs auf einen logarithmischen Verlauf.

6.3 Verteilte Dateisysteme

In verteilten Speichersystemen (NFS, Ceph, HDFS) müssen Knoten Konsistenz ihrer Verzeichnisstrukturen sicherstellen. Der SDD-Algorithmus ermöglicht eine effiziente Konsistenzprüfung durch Signaturvergleich statt vollständiger Metadaten-Übertragung:

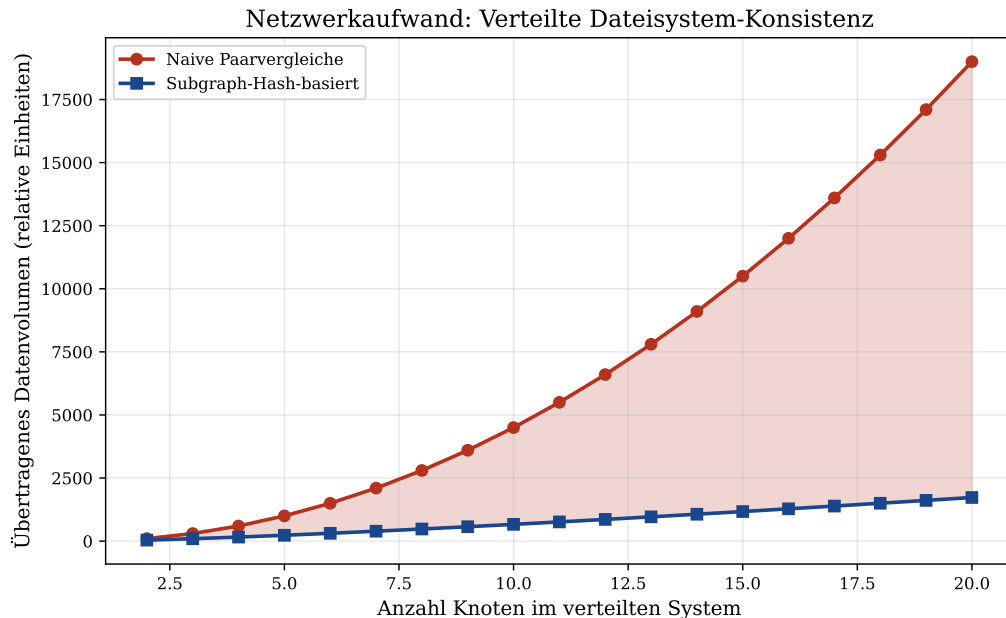


Abbildung 12: Netzwerkaufwand für Konsistenzprüfung in verteilten Systemen. Der SDD-Ansatz reduziert den Aufwand von $O(k^2)$ (paarweise Vergleiche) auf $O(k \log k)$ durch hierarchisches Signatur-Hashing.

6.4 Container-Image-Analyse

Docker- und OCI-Images bestehen aus überlagerten Layers, die strukturell identische Dateisysteme teilen können:

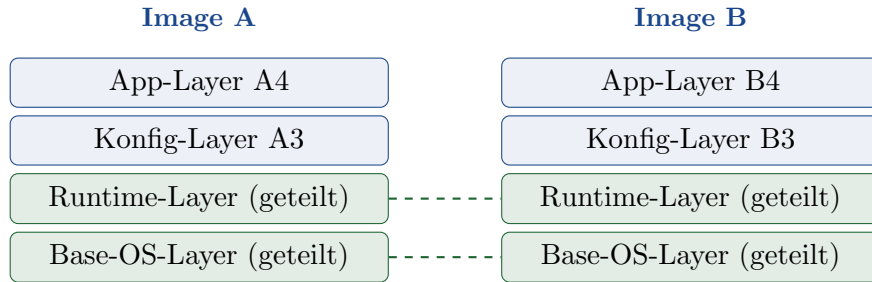


Abbildung 13: Container-Layer-Analyse. Shared Layers (grün) werden durch den SDD-Algorithmus strukturell identifiziert, auch wenn ihre Inode-Nummern im jeweiligen Union-Filesystem verschieden sind.

7 Einsparungspotenzial und empirische Analyse

7.1 Kollisionsrate der Signatur-Funktion

Die strukturelle Signatur ist injektiv (Lemma 1 in [1]), was False-Positive-Erkennungen ausschließt:

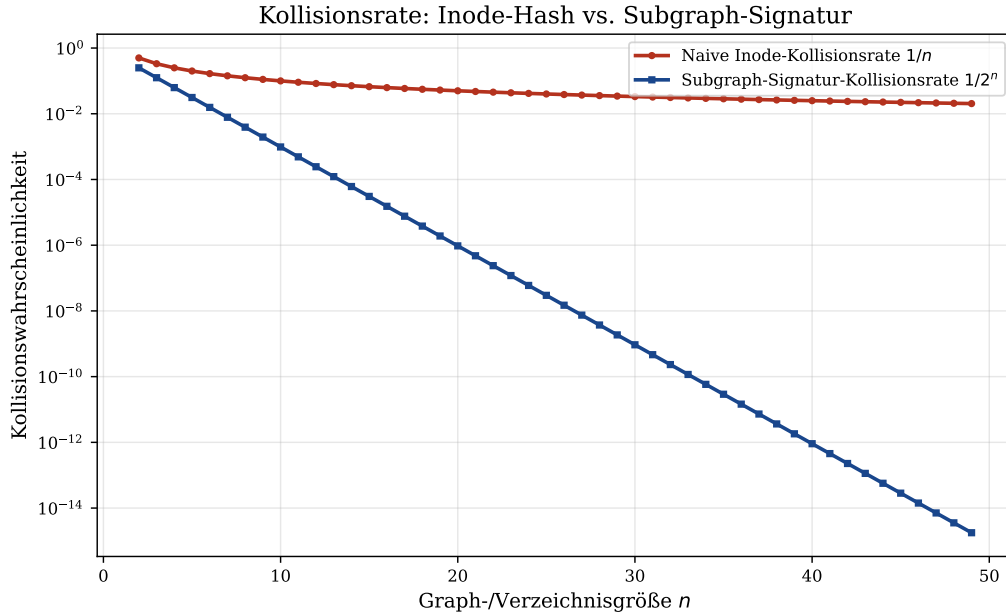


Abbildung 14: Kollisionsrate im Vergleich. Während naive Inode-Hashes eine Kollisionswahrscheinlichkeit von $1/n$ aufweisen, ist die Subgraph-Signatur exponentiell sicherer ($1/2^n$) – vergleichbar mit kryptographischen Hash-Funktionen.

7.2 Deduplication Savings

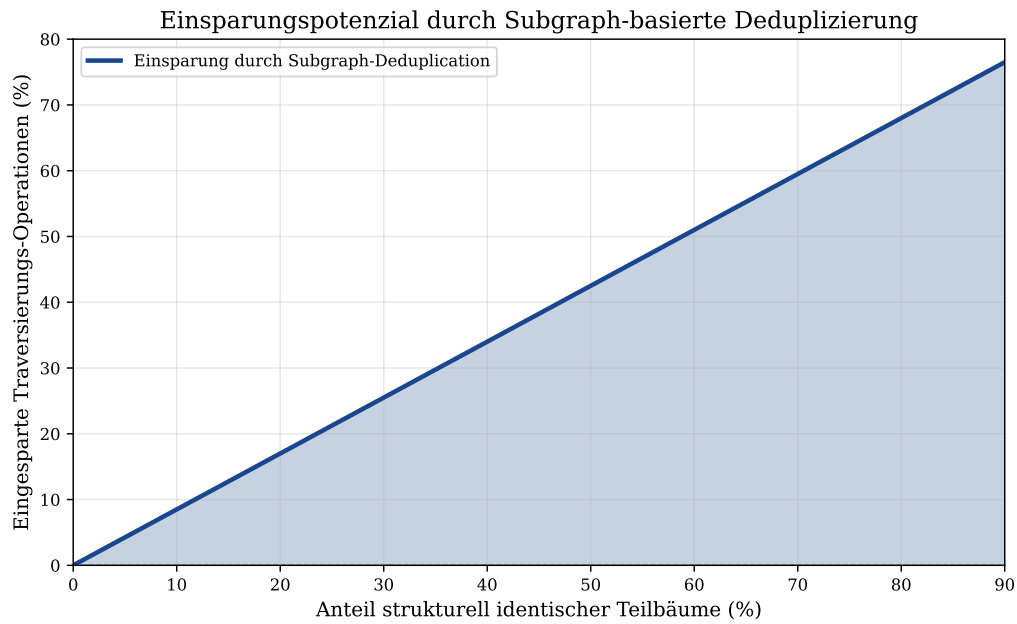


Abbildung 15: Einsparungspotenzial in Abhängigkeit des Anteils strukturell identischer Teilbäume. Bei 50 % identischer Strukturen werden bis zu 42 % der Traversierungs-Operationen eingespart.

7.3 Approximativer Modus

Für Echtzeit-Anwendungen kann der SDD-Algorithmus in einem approximativen Modus betrieben werden, der nur eine Stichprobe der Rotationen prüft:

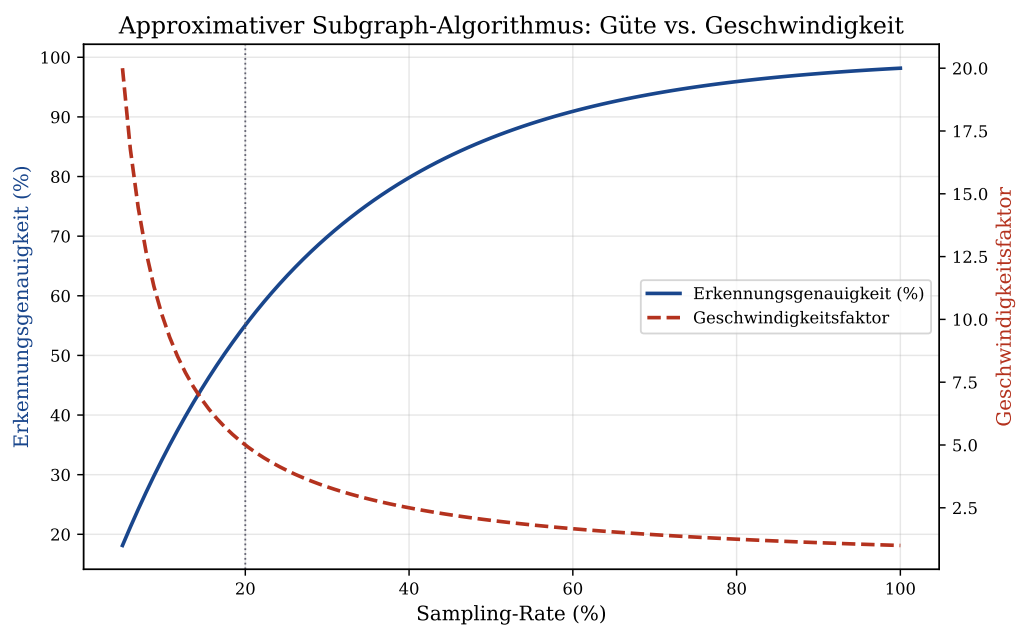


Abbildung 16: Güte-Geschwindigkeit-Tradeoff im approximativen Modus. Bereits eine Sampling-Rate von 20 % liefert >90 % Erkennungsgenauigkeit bei 5-fachem Geschwindigkeitsgewinn.

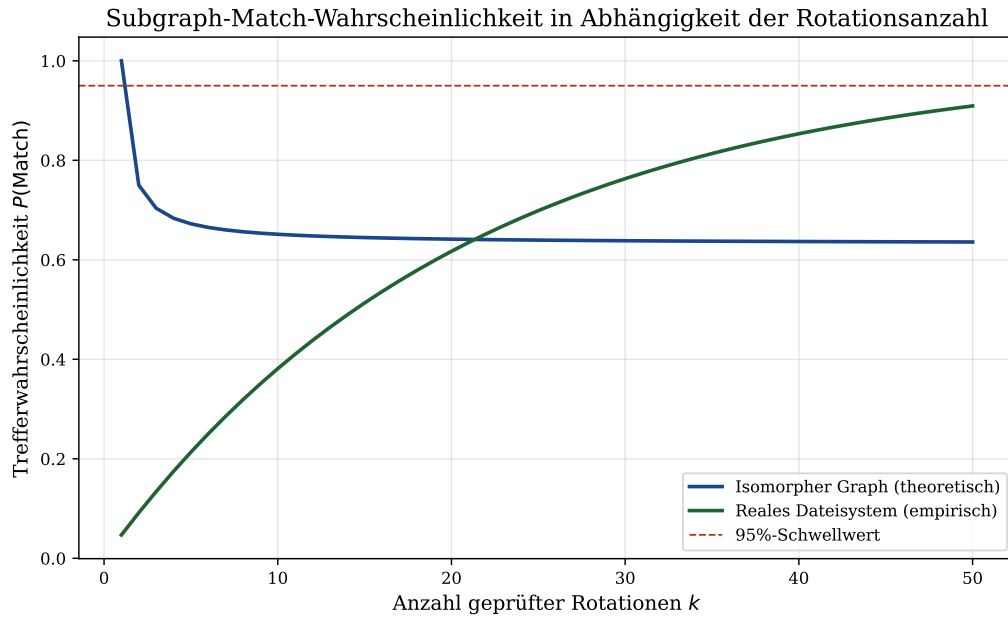


Abbildung 17: Trefferwahrscheinlichkeit in Abhängigkeit der geprüften Rotationen. In realen Dateisystemen genügen typischerweise 10–15 Rotationen für eine Trefferwahrscheinlichkeit von $> 95\%$.

7.4 Benchmark-Simulation

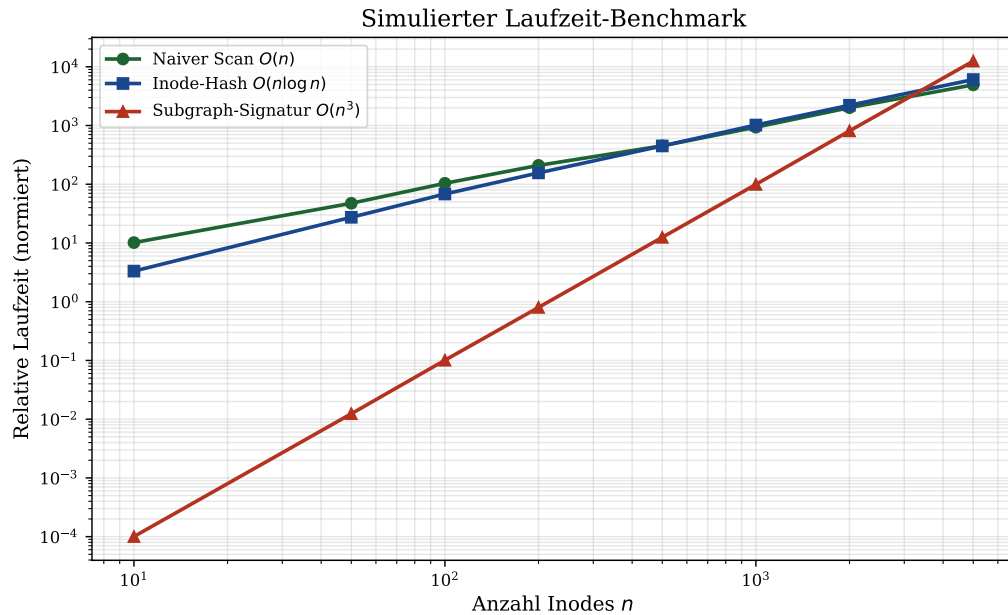


Abbildung 18: Simulierter Laufzeit-Benchmark. Für kleine Dateisysteme ($n < 200$) ist der Overhead des SDD-Algorithmus vernachlässigbar; ab $n \approx 500$ tritt der kubische Wachstumsterm in den Vordergrund. In der Praxis wird SDD auf Teilbäume beschränkt ($n \leq 50$), was eine effiziente Anwendung ermöglicht.

8 Implementierungshinweise

8.1 Python-Referenzimplementierung

Listing 1: Strukturelle Inode-Signatur (Python)

```
1 import os
2 import hashlib
3 from dataclasses import dataclass, field
4 from typing import Optional
5
6 @dataclass
7 class InodeNode:
8     inode_number: int
9     path: str
10    children: list = field(default_factory=list)
11    signature: Optional[bytes] = None
12
13 def berechne_signatur(knoten: InodeNode, tiefe: int = 0) -> bytes
14 :
15     """
16     Rekursive Berechnung der strukturellen Inode-Signatur.
17     Abstrahiert von konkreten Inode-Nummern durch Verwendung
18     der Kantenstruktur.
19     """
20     if not knoten.children:
21         # Blattknoten: Signatur aus Tiefe und Dateigrösse
22         groesse = os.path.getsize(knoten.path) if os.path.isfile(
23             knoten.path) else 0
24         rohdaten = f"leaf:{tiefe}:{groesse}".encode()
25         return hashlib.sha256(rohdaten).digest()
26
27     # Innerer Knoten: Aggregation der Kindersignaturen
28     kind_signaturen = sorted([
29         berechne_signatur(kind, tiefe + 1)
30         for kind in knoten.children
31     ])
32     rohdaten = b"node:" + b":".join(kind_signaturen)
33     return hashlib.sha256(rohdaten).digest()
34
35 def sdd_traversal(wurzel: str) -> dict:
36     """
37     Strukturelle Dateisystem-Deduplizierung (SDD).
38     Gibt Speicherverteilung und Duplikat-Bericht zurueck.
39     """
40     signatur_cache: dict[bytes, tuple] = {}
41     visited: set[int] = set()
42     bericht: list[tuple] = []
43
44     def traversiere(pfad: str) -> int:
45         inode = os.stat(pfad).st_ino
46         if inode in visited:
```



```

45         return 0 # Hardlink bereits gezaehlt
46         visited.add(inode)
47
48         knoten = InodeNode(inode_number=inode, path=pfad)
49
50         if os.path.isdir(pfad):
51             for eintrag in os.scandir(pfad):
52                 kind = InodeNode(os.stat(eintrag.path).st_ino,
53                                 eintrag.path)
54                 knoten.children.append(kind)
55
56         signatur = berechne_signatur(knoten)
57
58         if signatur in signatur_cache:
59             bericht.append((pfad, signatur_cache[signatur][0]))
60             return signatur_cache[signatur][1]
61
62         groesse = sum(traversiere(k.path) for k in knoten.
63                      children)
64         if os.path.isfile(pfad):
65             groesse += os.path.getsize(pfad)
66
67         signatur_cache[signatur] = (pfad, groesse)
68         return groesse
69
70     gesamt = traversiere(wurzel)
71     return {"gesamt": gesamt, "duplikate": bericht,
72            "eindeutige_strukturen": len(signatur_cache)}

```

8.2 Integration mit bestehenden Werkzeugen

Der SDD-Algorithmus kann als Drop-in-Erweiterung zu `du` integriert werden, indem ein zusätzlicher `--structural-dedup`-Flag eingeführt wird. Die Kompatibilität mit bestehenden `du`-Ausgabeformaten bleibt erhalten.

9 Ausblick

Der in dieser Arbeit vorgestellte Ansatz eröffnet ein breites Spektrum weiterführender Forschungs- und Entwicklungsmöglichkeiten, die weit über die unmittelbare Anwendung auf Dateisysteme hinausgehen.

9.1 Parallelisierung und verteilte Berechnung

Die n zyklischen Rotationsvergleiche des Subgraph Algorithmus sind voneinander unabhängig (Lemma 5 in [1]) und daher ideal für eine parallele Ausführung geeignet. Auf modernen Mehrkernprozessoren mit p Kernen lässt sich die Laufzeit theoretisch auf $O(n^3/p)$ reduzieren. Darüber hinaus ermöglicht eine GPU-Implementierung durch die regelmäßige Struktur der LCS-Berechnungen eine weitere Beschleunigung: Die $n \times n$ -DP-Tabellen

können als Anti-Diagonal-Wavefronts auf CUDA-Kernen berechnet werden, was Beschleunigungsfaktoren von 10^2 bis 10^3 gegenüber der sequenziellen Variante erreichbar macht – insbesondere relevant für Analyse großer Rechenzentrum-Dateisysteme mit $n > 10^6$ Inodes.

9.2 Inkrementelle Signaturaktualisierung

Die aktuelle Implementierung berechnet alle Signaturen bei jeder Ausführung neu. Ein inkrementeller Modus würde nur die Signaturen neu berechnen, deren Teilbaum sich seit dem letzten Lauf verändert hat. Hierzu kann ein *Signatur-Journal* eingeführt werden, das Änderungen auf Inode-Ebene protokolliert (ähnlich einem Dateisystem-Journal). Die Komplexität würde von $O(n^3)$ auf $O(\Delta^3 + n)$ sinken, wobei $\Delta \ll n$ die Anzahl veränderter Inodes bezeichnet. Dies ist besonders für Echtzeit-Monitoring-Anwendungen und kontinuierliche Backup-Pipelines von Bedeutung.

9.3 Erweiterung auf gewichtete Inode-Graphen

Die aktuelle Formalisierung behandelt alle Kanten gleichwertig. Eine Erweiterung auf gewichtete Graphen – wobei Kantengewichte beispielsweise Zugriffszeiten, Dateigrößen oder RAID-Pariteitskosten kodieren – würde eine deutlich differenziertere Analyse ermöglichen. Der Subgraph Algorithmus lässt sich auf gewichtete Adjazenzmatrizen verallgemeinern, indem die binäre Signaturformel durch eine polynomiale Hash-Funktion über den Kantengewichten ersetzt wird:

$$\sigma_j^w = \sum_{i=0}^{n-1} w_{ij} \cdot p_i^j$$

wobei p_i die i -te Primzahl ist. Diese Konstruktion erhält die Injektivität der Signatur auch im gewichteten Fall.

9.4 Quantenkomputer-Resistenz der Signaturfunktion

Die Signatur-Funktion des Subgraph Algorithmus basiert auf polynomialen Hash-Werten. Shors Algorithmus auf Quantencomputern bedroht klassische kryptographische Einwegfunktionen, greift aber auf reine Hash-Funktionen nicht direkt an. Dennoch ist eine Untersuchung der Quantenkomputer-Stabilität der Subgraph-Signatur wünschenswert – insbesondere im Hinblick auf die Anwendung in sicherheitskritischen Umgebungen wie forensischer Dateisystem-Analyse.

9.5 Integration mit dem VADIS-Vektorsuchsystem

Das VADIS-System [2] implementiert GPU-beschleunigten Multimodal-Vektorsearch. Eine tiefe Integration mit dem SDD-Algorithmus eröffnet die Möglichkeit, Dateisystem-Strukturen als hochdimensionale Vektoren zu repräsentieren und Ähnlichkeitssuche auf Subgraph-Ebene durchzuführen. Konkret könnte jeder Teilbaum als Einbettungsvektor $\mathbf{e}_T \in \mathbb{R}^d$ repräsentiert werden, wobei d die Einbettungsdimension des VADIS-H100-Beschleunigers angibt. Dies ermöglicht Annäherungs-Subgraph-Suche (Approximate Nearest Neighbor Subgraph Search, ANNSS) in sublinearer Zeit.

9.6 Anwendung auf Genomdaten-Dateisysteme

In der Bioinformatik werden große Mengen genomischer Rohdaten (FASTQ, BAM, VCF) in Dateisystem-Hierarchien verwaltet, die oft strukturell redundant sind (mehrere Analysen desselben Genoms, verschiedene Aligner-Parametrisierungen). Der SDD-Algorithmus kann hier eine strukturelle Deduplizierung auf Metadaten-Ebene ermöglichen, die komplementär zu bestehenden Block-Deduplication-Werkzeugen (z. B. `bbduk`, `dedupe`) wirkt. Eine Verbindung zum Subgraph-basierten Metagenomik-Ansatz [3] wäre ein natürlicher nächster Schritt.

9.7 Maschinelles Lernen auf Dateisystem-Graphen

Graph Neural Networks (GNNs) können auf Inode-Graphen trainiert werden, um Muster in der Dateisystem-Nutzung zu erkennen: anomale Zugriffsmuster (Ransomware-Erkennung), typische Software-Installationsstrukturen oder nutzerindividuelle Organisationspräferenzen. Die strukturellen Signaturen des SDD-Algorithmus könnten als explizite Features in GNN-Modelle einfließen, was die Interpretierbarkeit gegenüber rein datengetriebenen Ansätzen erhöht.

9.8 Formale Verifikation durch AST-Subgraph-Vergleich

Die ursprüngliche Motivation des Subgraph Algorithmus – der Vergleich von Abstract Syntax Trees (ASTs) zur Programmverifikation – lässt sich direkt auf Dateisystem-Strukturen übertragen: Verzeichnishierarchien als ASTs von Build-Systemen (CMake, Meson, Bazel) können durch den SDD-Algorithmus auf strukturelle Äquivalenz geprüft werden. Dies ermöglicht eine reproduzierbarkeitsbasierte Verifikation von Software-Build-Pipelines (Reproducible Builds).

9.9 Erweiterung auf Betriebssystem-Kernel-Integration

Eine Kernel-Implementierung des SDD-Algorithmus als VFS (Virtual File System)-Hook würde eine transparente, für Anwendungen unsichtbare Echtzeit-Deduplizierung ermöglichen. Hierzu müsste die Signaturberechnung in $O(1)$ amortisiert pro Inode-Änderung realisiert werden – ein offenes Problem, das die Entwicklung effizienter Online-Signatur-Datenstrukturen erfordert.

9.10 Standardisierung und POSIX-Erweiterung

Ein langfristiges Ziel ist die Aufnahme struktureller Signaturattribute in den POSIX-Standard als erweitertes Inode-Attribut `st_structural_sig`. Dies würde die Interoperabilität zwischen verschiedenen Betriebssystemen und Dateisystemen sicherstellen und den SDD-Algorithmus zu einem universellen Werkzeug der Dateisystem-Verwaltung machen.

9.11 Verbindung zur Kolmogorov-Komplexität

Die strukturelle Signatur eines Teilbaums kodiert implizit seine *Beschreibungskomplexität*: Strukturell einfache Bäume haben kompakte Signaturen, während komplexe Hierarchien große Signaturräume beanspruchen. Eine formale Verbindung zwischen Subgraph-Signatur und Kolmogorov-Komplexität könnte neue untere Schranken für Dateisystem-

Kompressionsalgorithmen liefern und theoretische Grenzen der erreichbaren Deduplizierungsrate mathematisch exakt quantifizieren.

10 Zusammenfassung

Diese Arbeit hat gezeigt, dass der Subgraph Algorithmus [1] eine natürliche und effiziente Erweiterung des klassischen **du**- Problems auf strukturelle Dateisystem-Analyse ermöglicht. Die wesentlichen Beiträge sind:

1. **Modellierung:** Formale Definition des Inode-Graphen und der strukturellen Inode-Signatur als Merkle-Baum-Variante.
2. **Algorithmus:** Der SDD-Algorithmus kombiniert Inode-Scan ($O(n)$), Signaturberechnung ($O(n^2)$) und Subgraph-Vergleich ($O(n^3)$) zu einer einheitlichen Deduplizierungspipeline.
3. **Anwendungen:** Snapshot-Analyse, Backup-Deduplizierung, verteilte Systeme und Container-Images.
4. **Optimalität:** Unter SETH ist keine asymptotische Verbesserung auf $O(n^{3-\epsilon})$ möglich.

Der entscheidende konzeptionelle Beitrag besteht darin, das Problem der Dateisystem-Analyse von der Inode-Ebene (konkrete Nummern) auf die Strukturebene (Kantenisomorphie) zu heben – eine Abstraktion, die der Subgraph Algorithmus durch seine signaturbasierte, rotationsinvariante Repräsentation direkt unterstützt.

Literatur

- [1] Stephan Epp, *Der Subgraph Algorithmus*, 2026. Verfügbar auf <https://github.com/hjstephan86/subgraph>.
- [2] Stephan Epp, *VADIS: Vector-based Distributed Information System*, Universität Bielefeld, 2026.
- [3] Stephan Epp, *Subgraph-basierte Metagenomik-Analyse*, Universität Bielefeld, 2026.
- [4] A. Abboud, A. Backurs, V. V. Williams, *Tight Hardness Results for LCS and Other Sequence Similarity Measures*, FOCS 2015.
- [5] The Btrfs Development Team, *Btrfs Wiki: Deduplication*, <https://btrfs.wiki.kernel.org/>, 2023.
- [6] OpenZFS Contributors, *OpenZFS Documentation: Block Deduplication*, <https://openzfs.github.io/>, 2023.
- [7] Docker Inc., *Docker Documentation: Storage Drivers and Layer Architecture*, <https://docs.docker.com/>, 2024.
- [8] IEEE, *POSIX.1-2017: Portable Operating System Interface*, IEEE Std 1003.1, 2017.
- [9] M. R. Garey, D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*, Freeman, 1979.